Scale — The General IR

Purpose. One portable intermediate representation carries every abstraction that must cross a Scale host boundary: expressions, patterns, declarations, kinds, predicates, sorts, conditions, dispatch, and actor messages. Pattern matching is not a separate wire format; it is the first typed sub-language of this IR.

Why one IR. A host-native surface is sugar. Elixir macros, F# quotations, and the prelude all elaborate to the same nodes, so a package, a rule, or a code-carrying actor message means the same thing on every host. The alternative—one portable format per feature—duplicates versioning, compatibility, and failure reporting.

Status. The runtime pattern IR and its closed portable document are implemented in `pattern-ast.fs` and `pattern-portable.fs`. The matcher and the `match` surface are the F# reference in `components/prelude/evaluator/pattern.fs`. The general expression, package, condition, protocol, and actor-message nodes are the target model below, not yet implemented.

1. Four representations

The same program passes through four representations. Only the third crosses a host boundary.

Representation	Role
Host surface	Native syntax: F# pattern syntax, Elixir macros, the general embedded surface, the prelude surface.
Elaborated IR	Typed, constructor-headed, host-independent. This is what analyses read and what the kernel executes.
Portable form	Canonical, versioned serialization of the IR plus its registries. This is what a package or a hot-reload unit carries.
Runtime image	Values are tensors in the append-only image. The IR is code; the image is data. They are never the same representation.

The host surface may keep host-specific features. The elaborated IR and the portable form may not.

D160 refines the elaborated IR into the typed term list: the runtime `Program` is a `Term list`, a quotation is a `Quote of Term list`, and `apply` splices. The term list is the execution representation; the lowered instruction stream is a compilation target. The current implementation still uses token text; the migration and the kernel/Prelude split are in `docs/term-ir.typ`.

2. What the IR contains

2.1 Values and terms

Terms are the runtime shape unification already uses: variables, atoms, compounds, typed terms, and lambdas. A pattern is a term restricted to constructor heads and the four executable variable roles. The IR distinguishes a **term** (a value under unification) from an **expression** (code that produces a value).

2.2 Expressions and quotations

Expression nodes cover literals, word references, calls, quotations, blocks, `let`, `lambda`, `if`, `apply`, and `define`. A quotation is an expression node, so a code-carrying actor message is an ordinary value of the IR. The pattern sub-language is a typed projection of the constructor and literal nodes; it never contains a function.

2.3 Patterns

The executable sub-language is the current pattern IR: wildcard, variable with `fresh`, `pin`, `repeated`, or `shadowing`, numeric, text, atom, constructor, and typed pattern. Clauses add a guarded or unguarded marker, a named predicate reference, a body expression, and a preserved order. A clause failure is structural or guard-rejected; `NoClauseMatched` is the per-match composition.

2.4 Declarations, modules, and packages

A package carries an identity, a schema version, a source hash, imports, entry points, and versioned code units. A code unit is the hot-reload boundary: it names its declared kinds and predicates so a loader can refuse a stale reference instead of resolving it silently. The prelude is one package loaded as a compiled image, not as host source text.

2.5 Kinds and constructors

Kinds are canonical registrations: a name, an arity, field presence, segment specs, closed or open status, and whether a host-extensible failure class covers. A pattern constructor refers to a kind, never to a host type identifier. The kind registry is the common vocabulary for patterns, data constructors, sorts, and protocols.

2.6 Predicates and guards

A predicate is a named reference with a pinned version and metadata: purity, coverage, and whether it may observe the host. A portable guard is a named predicate; a host-expression guard is host-marked and is not portable. A guard declaration says whether a false result covers the pattern for exhaustiveness.

2.7 Sorts and types

The existing set-theoretic model remains the authority: a sort is a store-resident value set with a name and an extension; a typed term carries a sort name; unification and predicate sorts decide membership. The IR records the sort annotations and the declarations a host must register. Host types map through the adapter's `tag` and `construct`; they do not become IR nodes.

2.8 Conditions, restarts, and handlers

Conditions are image-resident values; restarts are image-resident values with a name and a handler quotation; handler frames are execution state. The IR carries the condition constructor, the restart reference, and the handler body. If a handler crosses a host boundary, it becomes a portable quotation with its condition and restart names pinned; if it stays local, the adapter maps the host exception or exit to the same condition value.

2.9 Dispatch and protocols

Generic functions, methods, qualifiers, classes, and standard method combination are already store-resident data. The IR carries the declaration forms and the canonical names; the dispatch algorithm itself remains a Prelude concern. A protocol or behaviour declaration is a set of method signatures with a kind name and a dispatch policy, so it round-trips without a host interface type.

2.10 Actors, messages, and supervision

The actor model is Scale-defined: mailboxes, links, monitors, selective receive, traps, system messages, restart policies, and deadlines. The IR carries message constructors, message schemas, and supervision policy values. It does not carry a host process identifier; `EntryIdentity` and the actor capability are the portable references. A message that carries code carries an expression node, not a host closure.

2.11 Effects and ownership

Ownership is execution state, not data. The IR carries the linear-contract annotations and the reclamation intent that a package declares; the kernel's ownership checks execute them. No expected value, capability, or lifetime is inferred from a host reference.

3. Three axes, not one

Every node is classified on three axes:

- **Phase**: surface, elaborated, portable, loaded, executing.
- **Portability**: portable, host-marked, or host-only. A host-marked feature (bit syntax, open maps, active patterns, type providers, host expression guards, host exception types) is legal at the surface and illegal in the portable form.
- **Execution**: decision tree for ground subjects, unifier for open or equation terms, or host adapter for an opaque predicate. The interface defines the observation; the engine is an implementation choice.

4. Node sketch

The reference IR is a closed discriminated union. This is a shape sketch, not a frozen schema:

```
type Node =
    | Literal of value: Literal
    | Word of name: string
    | Call of target: Node * arguments: Node list
    | Quotation of body: Node list
    | Block of body: Node list
    | Let of name: string * value: Node * body: Node
    | Lambda of parameters: string list * body: Node
    | If of condition: Node * thenBranch: Node * elseBranch: Node
    | PatternClause of pattern: Pattern * guard: GuardRef option * body: Node
    | DeclareKind of PatternKind
    | DeclarePredicate of PredicateKind
    | Condition of name: string * message: Node
    | Restart of name: string * handler: Node
    | Generic of name: string * arity: int * methods: Node list
    | Message of kind: string * fields: Node list
```

The portable form is a tagged JSON tree with the same tags, a schema version, and the registries the nodes reference. Values are never embedded as host objects; a literal is a tensor or text entry in the image.

5. Package and version model

A package document has:

- `schema`: the IR schema version;
- `package`: identity, version, and source hash;
- `requires`: pinned kind, predicate, sort, and condition names;
- `units`: ordered code units, each with its own source hash and dependencies;
- `entry`: the package's exported names and entry quotations.

The loader checks the schema and every pinned reference before it executes a node. A new predicate with an old pattern is refused if the pattern pins an older version; a new pattern with an old predicate is refused the same way. Hot reload replaces a unit only when that unit's dependencies and pinned names still resolve.

6. Host adapter contract

Six functions cross the boundary. Nothing else:

`tag`	Classify a host term as a registered kind.
`extract`	Match a host term's shape and return bindings or an extraction failure.
`evaluate_guard`	Interpret a named predicate or a host-opaque guard.
`construct`	Build a host term from a kind and arguments.
`raise_failure`	Raise or report a typed failure class.
`query_kind_metadata`	Answer closed, arity, field-presence, and segment questions for analyses.

The adapter may refine a host feature, but it may not invent a portable node. Unknown roles, guards, kinds, or predicates are load-time errors, not opaque runtime constraints.

7. Portability boundaries

Feature	Status	Rule
Patterns, conditions, dispatch declarations, actor messages	Portable	Elaborate to the IR; serialize canonically.
Bit syntax, open maps, active patterns, type providers	Host-marked	Allowed at the host surface, refused in the portable form.
Host expression guards	Host-marked	A portable guard must be a named predicate with a pinned version.
Host exception and exit types	Host-marked	Map to a Scale condition; never serialize the host type.
Host process identifiers and native closures	Host-only	Replace with `EntryIdentity`, actor capabilities, and IR quotations.

8. Invariants

- The portable form is closed. No unknown role, guard, kind, or predicate survives loading.
- Patterns are constructor-headed. Functions never cross as patterns.
- Clause order is preserved, and `NoClauseMatched` is composed per match.
- Kinds, predicates, and condition names are versioned and pinned.
- A guard is a named predicate or a host-marked opaque call, never an arbitrary expression in the portable form.
- The IR is code; the image is data. Values are tensors and text entries.
- Serialization is explicit `Utf8JsonWriter` and `JsonDocument` code, never reflection over F# records.
- A host adapter interprets; it does not add semantics to the portable subset.

9. Staged plan

- **Phase 1, landed**: runtime pattern IR, closed portable pattern document, ground matcher, open-subject unifier path, and the `match` surface.
- **Phase 2, next**: general expression and quotation nodes, with patterns as their typed sub-language.
- **Phase 3**: package/module documents and pinned registries.

- **Phase 4**: conditions, restarts, dispatch declarations, and actor message schemas.
- **Phase 5, partially landed**: best-effort reachability, redundancy, and closed-kind exhaustiveness over the pattern IR (`pattern-analysis.fs`); the general-expression analyses remain open.
- **Phase 6**: the host adapters and the prelude bootstrap.

10. Current state

The implemented pieces are the pattern IR (`pattern-ast.fs`), the best-effort analyses (`pattern-analysis.fs`), the closed portable pattern document and the separate open template document (`pattern-portable.fs`), and the reference matcher and `match` word (`components/prelude/evaluator/pattern.fs`). The general expression AST is `kernel/expression-ast.fs` and its canonical JSON v1 document is `kernel/expression-portable.fs`; the package/unit document is `kernel/package-ast.fs` and `kernel/package-portable.fs`. The condition and protocol nodes and actor-message schemas are design targets. Nothing in this document authorizes a second portable format.

11. Engine contract: decision tree and unifier

Decision. The IR fixes observable results, failure classes, clause order, guard timing, and binding commitment. It does not mandate an engine. Ground subjects use the decision-tree engine; open or equation subjects use the unifier. On the intersection—a ground subject and a closed pattern—both engines must return the same result, the same failure class, and the same bindings.

	Decision tree	Unifier
Subject	A ground value	An open or equation term, possibly with variables
Operation	Match and bind	Solve equations and bind
Failure	Structural mismatch	Structural mismatch or `Undecided`
Higher-order unification	Not applicable	Miller patterns, flex-flex, occurs-check
Performance	Compiled pattern matrix with shared tests	Dynamic substitution; can grow quickly and is incomplete
Commit	Local until the clause succeeds	Speculative until unified

Our split. F# and Elixir host matching is mostly the ground case. Scale higher-order unification, typed terms, and open equations are the unifier case. Prelude dispatch and actor-message matching stay on the decision-tree path. The current `matchPatternGround` is the seed of the decision-tree engine; the current `openMatch` path is the unifier.

Conformance. Clause order is preserved. A guard runs only after a complete structural match and before any binding is committed. A structural failure is reported before a guard is considered. A decision-tree engine that cannot handle an open subject returns `Undecided`; it never returns `false`.

Why not one engine. A decision tree is the hot, predictable path, but it does not solve equations. A unifier is the general path, but it is unnecessarily expensive and incomplete for ground dispatch. Keeping both behind one interface gives host speed without giving up Scale's unification semantics.

12. Types, inference, and hints

Decision. The portable IR carries semantic type references, not inferred host types. A `TypeRef` names a sort or value set and is part of correctness. Inferred facts are optional unit-level hints and are never trusted for semantics.

Why. Inference is a host and elaborator concern. A host-inferred type can be useful for specialization, but it is not a portable fact; shipping it as semantic data would make one host's inference a requirement for another. A hint may also be wrong after a hot reload, so execution must remain correct when every hint is absent or stale.

Hint placement. Hints live in a unit-level table, not on every node. This keeps canonical equality and the semantic hash clean, lets a host omit the table, and lets the loader ignore unknown hint versions. A hint the kernel cannot validate is a fast-path candidate only; the checked path remains.

13. Versioning

Decision. There are five separate versions, not one.

Thing	What it covers	Change rule
IR schema	Node tags and fields	Major for incompatible shape; minor for additive nodes
Kind registry	Name, arity, closed/open, constructors, field and segment specs	Bump on any semantic change; content hash for exact identity
Predicate registry	Name, version, purity, coverage, host observability	Bump on behavior change; pin the exact version
Package	Identity, version, source hash, imports, capabilities	SemVer for dependencies; hash for the exact artifact
Unit	Source hash, dependencies, pinned kinds and predicates	The hot-reload boundary

Why package-local registries. A package must load without a mutable global registry. Package-local registrations plus a content hash make the package self-contained; a host-wide registry is an optional sharing layer, not a requirement. A name that two packages define with different content is a load-time conflict.

Pins. Kinds and predicates are pinned exactly. An old pattern with a new predicate, or a new pattern with an old predicate, is refused unless the package explicitly opts into a host-marked compatibility adapter. A hot-reload unit replaces another only when its exports, imports, and pins still resolve.

14. Trust boundary

Decision. The default is deny. A package declares requested capabilities; the host grants them. The IR carries the request, never the grant. Transport, filesystem, clock, random, host dispatch, and code execution are capabilities. A code-carrying actor message is data; executing it requires an execution grant.

Load-time checks. The loader enforces maximum serialized size, node count, nesting depth, and unit count. It refuses unknown tags, unknown roles, unknown guards, unknown kinds, and host objects. Reflection is not a loading path.

Why. Pattern-bearing code can arrive in a message. Without a capability boundary, a pattern package could become an unmediated host-call channel. Default-deny keeps the transport and the package model independent from the host's authority.

15. Source maps

Decision. A unit assigns node ids at elaboration and carries a separate source-map table: `nodeId -> file, line, column, length`. Node ids are non-semantic, excluded from the semantic hash and

from canonical equality. The portable form may omit the table; a package without one still loads, and diagnostics degrade to unit and position.

Why a side table. Inline spans bloat the portable form and complicate canonical equality. A side table keeps the semantic nodes clean, allows several source views (original, generated, transformed), and can be dropped from a release artifact without changing behavior. Transforms preserve ids or emit a rewrite table.

16. Prelude bootstrap

Decision. The prelude is portable-only: no host-marked nodes, no native closures, no host exception types, and no native process identifiers. It is one package with ordered units, a declared entry point, pinned kinds and predicates, and a declared capability set.

Sequence. First, freeze the core IR and the minimal kernel word set (D95). Second, write a minimal prelude in Scale: patterns, dispatch, actors, and conditions. Third, compile it to the general IR, canonical JSON, and the prelude image. Fourth, load and run it in the F# kernel against the conformance suite. Fifth, add the Elixir adapter and run the same package and oracle. Sixth, add hot reload and the old/new predicate and kind tests. Only then move the remaining prelude from F# reference code to Scale.

Why this order. The prelude cannot be the first thing ported: it needs the IR, the package model, the registries, and the adapter contract it will use. Conversely, the IR is not finished until the prelude is expressible in it. The two are the same milestone approached from opposite ends.

17. Locked decisions

- Partial portable core plus host-marked extensions.
- Canonical JSON v1 for the portable form.
- Ground subjects use the decision tree; open subjects use the unifier; no engine hint is required for correctness.
- Semantic `TypeRefs` are portable; inferred facts are optional unit-level hints, never trusted for semantics.
- Package-local kind and predicate registries with content hashes; an optional host-wide registry; exact pins.
- Default-deny capabilities; host dispatch and code execution are grants.
- Unit-scoped node ids plus an optional source-map table; ids are excluded from semantic equality.
- The prelude is portable-only, one package with units, F# adapter first, then Elixir.